
SCREENGUARD 2

Dossier d'Architecture Système
et d'Analyse Produit

Application Desktop de Protection Visuelle
Contextuelle de l'Écran

Version	2.0
Date	Avril 2026
Classification	Pré-étude produit / MVP
Porteur	Équipe ScreenGuard
Confidentialité	Usage interne

Ce document constitue un dossier de conception, d'architecture et d'analyse de conformité. Il ne se substitue à aucune validation juridique formelle ni à aucun audit de certification.

Table des matières

1	Résumé exécutif	4
2	Contexte et problématique	5
2.1	Risques de confidentialité visuelle	5
2.2	Cas d'usage réels	5
2.3	Limites des solutions existantes	5
3	Vision produit	6
3.1	Intention et promesse fonctionnelle	6
3.2	Cible utilisateur	6
3.3	Scénarios d'usage prioritaires	6
3.4	Valeur ajoutée	6
4	Objectifs du projet	7
4.1	Objectifs fonctionnels	7
4.2	Objectifs non fonctionnels	7
5	Périmètre du MVP	8
5.1	Ce que le MVP couvre	8
5.2	Ce que le MVP ne couvre pas	8
5.3	Arbitrages techniques	8
6	Cahier des charges détaillé	9
6.1	Exigences fonctionnelles	9
6.2	Exigences non fonctionnelles	9
7	Architecture globale du système	10
7.1	Vue d'ensemble	10
7.2	Vue en couches	10
7.3	Flux de données	10
7.4	Communication Python–JavaScript	10
8	Description détaillée des composants	11
8.1	Module de capture webcam	11
8.2	Détection faciale MediaPipe	11
8.3	Estimation du regard (GazeEstimator)	11
8.4	Encodage facial et reconnaissance	11
8.5	Score de suspicion et temporisation	11
9	Logique métier et machine à états	13
9.1	États du système	13
9.2	Transitions validées	13
9.3	Cas limites	13
10	Choix technologiques	14
11	UX/UI et design système	15
11.1	Philosophie de l'interface	15

11.2 Éléments fonctionnels vs décoratifs	15
12 Performance, robustesse et limites	16
12.1 Limites de détection	16
12.2 Faux positifs et faux négatifs	16
13 Sécurité, confidentialité et données personnelles	17
13.1 Architecture de traitement local	17
13.2 Inventaire des données	17
13.3 Conformité RGPD — analyse de principe	17
14 Normes, référentiels et analyse de conformité	19
15 Analyse des risques	20
16 Plan d'amélioration	21
16.1 Court terme (MVP+)	21
16.2 Moyen terme (v3.0)	21
16.3 Long terme (v4.0+)	21
17 Stratégie de validation	22
18 Conclusion	23

Chapitre 1

Résumé exécutif

ScreenGuard2 est une application desktop locale de protection visuelle contextuelle de l'écran. Elle exploite la webcam intégrée de l'ordinateur pour analyser en temps réel l'environnement visuel de l'utilisateur et détecter la présence éventuelle d'un observateur non autorisé. Lorsqu'un second visage est identifié comme orienté vers l'écran, le système déclenche automatiquement un overlay de protection qui masque le contenu affiché, préservant ainsi la confidentialité des informations sensibles.

Le projet se positionne comme un **MVP** (Minimum Viable Product) fonctionnel, modulaire et évolutif. Il repose sur un pipeline de vision par ordinateur combinant **OpenCV** pour la capture et le traitement d'images, et **MediaPipe Face Mesh** pour la détection de 468 points de repère faciaux incluant le suivi de l'iris. L'interface utilisateur adopte une esthétique premium inspirée des interfaces HUD aéronautiques, implémentée via PySide6 et QWebEngineView avec un frontend HTML/CSS/JavaScript.

Principes architecturaux fondamentaux

- Traitement 100 % local — aucune donnée transmise vers le cloud
- Aucun enregistrement vidéo — les frames sont volatiles
- Architecture modulaire en quatre couches indépendantes
- Machine à états formelle avec transitions validées
- Scoring progressif avec temporisation anti-faux positifs

Le système intègre une logique de scoring progressif, une machine à états formelle gérant les transitions entre les modes actif, protégé, partage temporaire et pause, ainsi qu'un mécanisme de verrouillage par code PIN pour les situations d'absence de l'utilisateur principal. La calibration faciale permet l'identification de l'utilisateur autorisé et le support de profils multi-utilisateurs.

Chapitre 2

Contexte et problématique

2.1 Risques de confidentialité visuelle

Dans les environnements professionnels contemporains — espaces de coworking, open spaces, transports, lieux publics — la confidentialité des informations affichées à l'écran constitue un risque opérationnel souvent sous-estimé. Les données sensibles (documents stratégiques, courriels confidentiels, données financières, informations clients, code source propriétaire) sont directement exposées au regard de toute personne située dans le champ visuel de l'écran.

Ce phénomène, désigné sous le terme de *shoulder surfing* (observation par-dessus l'épaule), représente un vecteur de fuite d'information particulièrement insidieux : il ne laisse aucune trace numérique et ne déclenche aucune alerte des systèmes de sécurité informatique traditionnels.

2.2 Cas d'usage réels

Les scénarios justifiant une protection visuelle contextuelle sont nombreux et concrets :

- Un consultant travaillant dans un espace client partagé, manipulant des données relevant du secret professionnel.
- Un développeur en open space dont l'écran affiche du code source propriétaire ou des clés d'API.
- Un responsable RH consultant des dossiers de personnel dans un bureau à cloisons vitrées.
- Un cadre dirigeant préparant un document stratégique dans un train ou un avion.
- Un médecin consultant un dossier patient en présence d'un tiers non autorisé.

2.3 Limites des solutions existantes

Les filtres de confidentialité physiques (films polarisants) limitent l'angle de vision latéral mais n'offrent aucune protection contre une observation frontale. Ils dégradent la luminosité et la fidélité colorimétrique de l'écran de manière permanente. Les solutions logicielles existantes sont majoritairement basées sur des approches cloud (envoi d'images à des serveurs distants pour analyse), ce qui pose des problèmes majeurs de confidentialité et de latence.

Paradoxe des solutions cloud

Les solutions de protection visuelle basées sur le cloud envoient précisément les images de l'écran et de son environnement à des serveurs tiers, créant un risque de confidentialité plus important que celui qu'elles prétendent résoudre.

Chapitre 3

Vision produit

3.1 Intention et promesse fonctionnelle

ScreenGuard2 vise à fournir une protection visuelle intelligente, contextuelle et non intrusive. La promesse fonctionnelle : l'écran de l'utilisateur est automatiquement protégé lorsqu'un observateur non autorisé est détecté, et cette protection est levée de manière transparente dès que la menace disparaît ou que l'utilisateur autorise explicitement le partage.

3.2 Cible utilisateur

Professionnels manipulant des données sensibles dans des environnements partagés : consultants, développeurs, juristes, responsables RH, cadres dirigeants, professionnels de santé. Secondairement, tout utilisateur soucieux de sa confidentialité visuelle.

3.3 Scénarios d'usage prioritaires

TABLE 3.1 – Scénarios d'usage prioritaires

Scénario	Déclencheur	Action système	Résultat attendu
Travail en open space	Collègue s'approche	Détection second visage, overlay activé	Contenu masqué en < 2 s
Présentation collègue	Clic « Partager 5 min »	Suspension temporaire	Écran visible, réactivation après délai
Pause, écran non verrouillé	Utilisateur quitte poste	Détection d'absence, verrouillage PIN	Écran verrouillé
Retour au poste	Visage principal re-détecté	Score décroît	Reprise normale

3.4 Valeur ajoutée

Par rapport aux filtres physiques, ScreenGuard2 offre une protection **contextuelle** (activée uniquement quand nécessaire), **sans dégradation permanente** de l'affichage, avec la possibilité de **partager ponctuellement** l'écran avec un collègue autorisé, et une **adaptation en temps réel** à la situation.

Chapitre 4

Objectifs du projet

4.1 Objectifs fonctionnels

TABLE 4.1 – Objectifs fonctionnels

ID	Objectif	Description
OF-01	Détection faciale multi-visages	Détecter jusqu'à 4 visages simultanément via MediaPipe Face Mesh (468 landmarks)
OF-02	Identification utilisateur principal	Distinguer l'utilisateur autorisé des tiers via encodage facial calibré
OF-03	Estimation du regard	Évaluer la direction du regard (yaw/pitch) via les landmarks de l'iris
OF-04	Détection d'observateur	Identifier un visage non autorisé orienté vers l'écran
OF-05	Protection visuelle instantanée	Activer un overlay opaque masquant l'intégralité du contenu
OF-06	Popup de décision	Proposer maintenir la protection ou autoriser un partage temporaire
OF-07	Partage temporaire	Suspendre la protection pour une durée choisie (30 s, 1 min, 5 min, 15 min)
OF-08	Réactivation automatique	Restaurer la protection à l'expiration du délai de partage
OF-09	Verrouillage par PIN	En cas d'absence, exiger un code PIN 4 chiffres
OF-10	Calibration faciale	Permettre l'enregistrement du profil facial de l'utilisateur principal

4.2 Objectifs non fonctionnels

TABLE 4.2 – Objectifs non fonctionnels

ID	Objectif	Description
ONF-01	Traitement 100 % local	Aucune donnée transmise, aucun service cloud, aucune API externe
ONF-02	Latence < 2 secondes	Délai maximum entre détection et activation de la protection
ONF-03	Consommation CPU < 15 %	En régime de scan à 500 ms
ONF-04	Modulaire et extensible	Architecture en couches séparées (vision, core, UI, config)
ONF-05	Packageable en .exe	Distribuable via PyInstaller sans installation Python
ONF-06	Configuration persistante	Tous les paramètres sauvegardés en JSON local

Chapitre 5

Périmètre du MVP

5.1 Ce que le MVP couvre

Le MVP couvre l'ensemble du pipeline fonctionnel de base : capture webcam à 640×480 , détection faciale via MediaPipe ou Haar Cascades en fallback, estimation heuristique du regard via les landmarks de l'iris, identification de l'utilisateur principal par encodage facial calibré, scoring de suspicion progressif, protection visuelle par overlay plein écran, popup de décision contextuelle, partage temporaire avec minuterie, verrouillage par code PIN en cas d'absence, interface JARVIS premium via QWebEngineView, configuration persistante en JSON, et logs rotatifs.

5.2 Ce que le MVP ne couvre pas

- Eye-tracking de précision infrarouge
- Reconnaissance faciale de niveau biométrique certifié
- Chiffrement des données de profil stockées
- Couverture multi-écrans complète
- Tests automatisés (unitaires, intégration, E2E)
- Accessibilité WCAG
- Internationalisation
- Signature numérique de l'exécutable
- Mode entreprise avec politique de groupe
- Certification de conformité RGPD par un DPO

5.3 Arbitrages techniques

Le choix de MediaPipe Tasks API plutôt qu'un modèle de deep learning custom se justifie par le compromis performance/précision/taille du modèle. Le fichier `face_landmarker.task` ne pèse que 4,7 Mo et fournit 468 landmarks faciaux incluant les 10 points de l'iris, suffisant pour une estimation heuristique de la direction du regard. Un modèle custom offrirait une précision supérieure mais nécessiterait un dataset d'entraînement conséquent et une taille de modèle incompatible avec l'objectif d'un exécutable léger.

Chapitre 6

Cahier des charges détaillé

6.1 Exigences fonctionnelles

TABLE 6.1 – Exigences fonctionnelles

ID	Exigence	Priorité
EF-01	Le système doit capturer des frames webcam à une fréquence configurable (défaut 500 ms)	Haute
EF-02	Le système doit détecter jusqu'à 4 visages par frame	Haute
EF-03	Le système doit distinguer l'utilisateur principal des observateurs via encodage facial	Haute
EF-04	Le système doit estimer la direction du regard de l'utilisateur principal	Moyenne
EF-05	Le système doit calculer un score de suspicion progressif	Haute
EF-06	Le système doit activer un overlay de protection lorsque le score dépasse le seuil	Haute
EF-07	Le système doit afficher une popup de décision	Haute
EF-08	Le système doit permettre un partage temporaire avec durées prédéfinies	Haute
EF-09	Le système doit réactiver automatiquement la protection après expiration	Haute
EF-10	Le système doit permettre la saisie d'un code PIN pour le déverrouillage	Moyenne
EF-11	Le système doit offrir un assistant de calibration faciale guidé	Moyenne
EF-12	Le système doit sauvegarder tous les paramètres en JSON local	Haute

6.2 Exigences non fonctionnelles

TABLE 6.2 – Exigences non fonctionnelles

ID	Exigence	Priorité
ENF-01	Aucune donnée image ne doit quitter la machine locale	Critique
ENF-02	Temps de réaction < 2 secondes entre détection et protection	Haute
ENF-03	Compatibilité Windows 10+, macOS 12+, Linux (X11)	Haute
ENF-04	Consommation mémoire < 500 Mo en régime normal	Moyenne
ENF-05	Packageable en exécutable autonome via PyInstaller	Haute
ENF-06	Interface lisible et utilisable sans formation préalable	Haute

Chapitre 7

Architecture globale du système

7.1 Vue d'ensemble

L'architecture de ScreenGuard2 repose sur une séparation stricte en quatre couches fonctionnelles indépendantes, communiquant via des signaux Qt asynchrones. Cette architecture modulaire permet le remplacement ou l'évolution de chaque composant sans impact sur les autres couches.

7.2 Vue en couches

TABLE 7.1 – Architecture en couches

Couche	Répertoire	Composants	Responsabilité
UI Layer	<code>ui/</code>	QWebView, HTML/CSS/JS JARVIS	Présentation, interaction utilisateur
Core Layer	<code>core/</code>	Engine, StateMachine, Logger	Logique métier, orchestration
Vision Layer	<code>vision/</code>	MediaPipe, GazeEstimator, FaceEncoder	Pipeline de vision par ordinateur
Config Layer	<code>config/</code>	Settings, ProfileManager	Persistance JSON, profils

7.3 Flux de données

Le flux de données suit un pipeline unidirectionnel : la webcam produit des frames brutes à une fréquence configurable. Chaque frame est traitée par le module de détection (MediaPipe ou Haar fallback) qui produit un objet `DetectionResult` contenant les landmarks faciaux, les bounding boxes, l'encodage facial du visage principal, l'estimation du regard, et le score d'attention. Le moteur (**Engine**) consomme ce résultat pour mettre à jour le score de suspicion et éventuellement déclencher une transition d'état.

7.4 Communication Python–JavaScript

Le pont bidirectionnel entre le backend Python et le frontend JavaScript est réalisé via `QWebChannel`. Un objet `Bridge` expose une méthode `on_command(cmd)` appelée par le JavaScript. Dans l'autre sens, le signal `send_to_js(cmd, payload)` permet à Python d'envoyer des commandes et des données JSON au frontend.

Chapitre 8

Description détaillée des composants

8.1 Module de capture webcam

La capture est assurée par OpenCV (`cv2.VideoCapture`). La résolution est fixée à 640×480 pour optimiser le compromis entre précision de détection et charge CPU. L'index de la caméra est configurable (défaut 0). La capture est déclenchée par un `QTimer` à intervalle configurable (défaut 500 ms). La caméra est libérée lorsque le système est désactivé.

8.2 Détection faciale MediaPipe

Le module utilise MediaPipe Face Landmarker (Tasks API) avec le modèle `face_landmarker.task` (float16, 4,7 Mo). Ce modèle fournit 478 landmarks par visage (468 points de maillage facial et 10 points d'iris). Le système gère automatiquement le téléchargement du modèle au premier lancement.

Gestion du modèle MediaPipe

Le module `model_manager.py` recherche le modèle dans l'ordre suivant : (1) cache local `~/.screenguard/models/`, (2) répertoire du package MediaPipe, (3) bundle PyInstaller. Si introuvable, il le télécharge automatiquement depuis Google Storage (4,7 Mo). En cas d'échec, le système bascule sur les Haar Cascades d'OpenCV.

8.3 Estimation du regard (GazeEstimator)

L'estimation du regard repose sur une heuristique combinant trois sources d'information : la géométrie faciale (ratio entre position du nez et milieu des yeux pour le yaw), le positionnement de l'iris (ratio de la position horizontale de l'iris dans l'ouverture oculaire), et la composante Z du landmark nasal. La fusion est pondérée : 60 % géométrie faciale + 40 % iris.

Limitation importante

Il s'agit d'une estimation heuristique, non d'un eye-tracking de précision. La fiabilité est estimée à environ 75–85 % dans des conditions d'éclairage normales, face à la caméra, sans lunettes de soleil.

8.4 Encodage facial et reconnaissance

L'encodage facial repose sur le calcul de 16 distances inter-landmarks normalisées. Ce vecteur de 16 dimensions est normalisé en norme L2. La comparaison utilise la similarité cosinus avec un seuil de 0,65. Cette approche n'est **pas** une biométrie certifiée.

8.5 Score de suspicion et temporisation

Le score de suspicion évolue selon les observations :

TABLE 8.1 – Règles de scoring

Événement	Incrément	Condition
Visage secondaire détecté	+1,5 / visage	Après délai anti-faux positifs (3 scans)
Absence utilisateur principal	+0,3 / scan	Après délai anti-faux positifs
Regard détourné de l'écran	+0,3 / scan	Yaw > 25° ou pitch > 20°
Utilisateur seul, regard centré	-0,8 / scan	Attention > 50 %

Le seuil de déclenchement est de **6** pour les observateurs et de **15** ($6 \times 2,5$) pour l'absence, rendant le déclenchement par absence environ **5 fois plus lent**.

Chapitre 9

Logique métier et machine à états

9.1 États du système

TABLE 9.1 – États du système

État	Description
IDLE	Système inactif, caméra libérée, aucun traitement
ACTIVE	Scan en cours, analyse des frames, scoring actif
PROTECTED	Overlay affiché, popup de décision ou PIN visible
SHARING	Protection temporairement levée, minuterie active
PAUSED	Scan suspendu (calibration en cours)
ERROR	Erreur caméra ou système

9.2 Transitions validées

TABLE 9.2 – Matrice de transitions

Source	Cible	Condition
IDLE	ACTIVE	Utilisateur active le système
ACTIVE	PROTECTED	Score de suspicion $\geq$ seuil
ACTIVE	PAUSED	Début de calibration
PROTECTED	ACTIVE	Rejet de l'alerte ou PIN validé
PROTECTED	SHARING	Partage temporaire accordé
SHARING	ACTIVE	Expiration du délai
PAUSED	ACTIVE	Fin de calibration
ERROR	IDLE	Réinitialisation manuelle
*	IDLE	Utilisateur désactive le système
*	ERROR	Perte de la caméra ou erreur critique

9.3 Cas limites

Si la caméra est déconnectée pendant un scan, le système transite vers ERROR. Si le modèle MediaPipe n'est pas disponible, le système bascule sur Haar Cascades. Si l'utilisateur ferme la popup sans répondre, la protection reste active. Si le score de suspicion atteint le seuil pendant un partage, le partage continue jusqu'à son expiration.

Chapitre 10

Choix technologiques

TABLE 10.1 – Justification des choix technologiques

Technologie	Rôle	Avantages	Limites
Python 3.12	Langage principal	Écosystème riche, prototypage rapide	Performance, taille .exe, GIL
OpenCV 4.9+	Capture et traitement	Standard industriel, Haar fallback	Dépendance lourde (~50 Mo)
MediaPipe 0.10+	Détection faciale, iris	478 landmarks, modèle 4,7 Mo, CPU rapide	API instable, Python 3.10–3.12
PySide6	Framework UI	Licence LGPL, QWebView	Poids (~80 Mo WebEngine)
HTML/CSS/JS	Frontend JARVIS	Design libre, animations 60 fps	Pas de framework réactif
PyInstaller	Packaging .exe	Standard, multi-OS	150–300 Mo, faux positifs AV

Chapitre 11

UX/UI et design système

11.1 Philosophie de l'interface

L'interface adopte une esthétique délibérément premium, inspirée des interfaces HUD des cockpits aéronautiques et des représentations cinématographiques de systèmes d'intelligence artificielle (JARVIS / Iron Man). Ce choix répond à plusieurs objectifs fonctionnels :

1. Le dark mode avec accents cyan réduit la fatigue visuelle.
2. La typographie monospace facilite la lecture instantanée des métriques.
3. Les cercles concentriques animés fournissent un feedback visuel constant de l'état du système.
4. Le point de regard sur le HUD central permet la vérification par l'utilisateur.

11.2 Éléments fonctionnels vs décoratifs

TABLE 11.1 – Classification des éléments d'interface

Élément	Fonction	Classification
Cercles orbitaux animés	Feedback d'état système	Fonctionnel
Point de regard sur HUD	Vérification détection	Fonctionnel
Jauge de suspicion	Niveau de menace temps réel	Fonctionnel
Badge d'état	État courant du système	Fonctionnel
Canvas de fond (lignes radiales)	Ambiance visuelle	Décoratif
Typographie Orbitron	Identité visuelle	Semi-fonctionnel

Chapitre 12

Performance, robustesse et limites

12.1 Limites de détection

TABLE 12.1 – Limites de détection identifiées

Scénario	Impact	Probabilité	Mitigation
Éclairage faible	Détection dégradée	Élevé	Seuils de confiance MediaPipe
Lunettes de soleil	Gaze tracking inopérant	Moyen	Fallback géométrie faciale
Angle extrême ($> 45^\circ$)	Visage non détecté	Moyen	Limitation modèles frontaux
Webcam basse résolution	Landmarks imprécis	Moyen	Résolution minimale 640×480
Occlusion partielle	Détection intermittente	Moyen	Temporisation (3 scans)
Photo affichant un visage	Détection comme vrai visage	Faible	Pas de liveness detection

12.2 Faux positifs et faux négatifs

Les faux positifs sont mitigés par la temporisation anti-faux positifs (3 scans consécutifs), le scoring progressif avec seuil à 6, et la possibilité de rejeter une fausse alerte. Les faux négatifs sont plus difficiles à mitiger : un observateur situé hors du champ de la webcam, portant des lunettes de soleil, ou partiellement masqué, ne sera pas détecté. Cette limitation est inhérente à tout système de vision non omnidirectionnel.

Chapitre 13

Sécurité, confidentialité et données personnelles

13.1 Architecture de traitement local

Aucune image capturée par la webcam n'est stockée sur disque, transmise à un serveur, ou conservée en mémoire au-delà du scan courant. Le flux vidéo est traité frame par frame : chaque frame est analysée, les résultats extraits, puis la frame est déréférencée.

13.2 Inventaire des données

TABLE 13.1 – Inventaire des données traitées et stockées

Donnée	Nature	Traitement	Persistance
Images webcam	Traitées en mémoire	Non stockées, non transmises	Volatile
Landmarks faciaux	Extraits pour analyse	Non stockés individuellement	Volatile
Encodage facial calibré	Vecteur 16 flottants	Stocké dans <code>profiles.json</code>	Persistant
Paramètres utilisateur	Seuils, préférences, PIN	Stocké dans <code>config.json</code>	Persistant
Logs applicatifs	Événements système	Stockés dans <code>screenguard.log</code>	Persistant (rotatif)

13.3 Conformité RGPD — analyse de principe

Avertissement

L'analyse ci-dessous reflète l'intention de conception mais ne constitue pas une validation juridique formelle.

TABLE 13.2 – Analyse de conformité RGPD

Article	Application au projet	Conformité
Art. 5 Minimisation	Le système ne collecte que les landmarks nécessaires. Aucune image stockée.	Conforme en intention
Art. 5 Conservation	Frames volatiles. Encodage facial persistant mais limitable.	Conforme en intention
Art. 6 Base légale	Intérêt légitime. Consentement implicite de l'utilisateur.	À documenter
Art. 13 Information	Pas de notice d'information formelle dans le MVP.	Non conforme
Art. 25 Privacy by Design	Architecture locale, pas de cloud, pas d'enregistrement.	Conforme
Art. 35 AIPD	Pas d'analyse d'impact formelle réalisée.	Non démontrable

Chapitre 14

Normes, référentiels et analyse de conformité

TABLE 14.1 – Analyse de conformité aux référentiels applicables

Référentiel	Domaine	Applicabilité	Mesures actuelles	Conformité	Écarts
ISO 9241-210	Conception centrée utilisateur	Bonne pratique	Calibration, feedback temps réel	Partielle	Pas de tests utilisateurs
ISO 27001	Sécurité de l'information	Recommandée	Traitement local	Faible	Pas de SMSI, pas d'audit
RGPD	Données personnelles	Applicable	Privacy by design	Partielle	Pas de notice, pas d'AIPD
ISO 25010	Qualité logicielle	Pertinente	Modularité	Partielle	Pas de tests auto
WCAG 2.1	Accessibilité	Souhaitable	Non implémentée	Non conforme	Pas de mode contraste
OWASP	Sécurité applicative	Recommandée	Local, pas de réseau	Partielle	PIN non haché

Chapitre 15

Analyse des risques

TABLE 15.1 – Registre des risques identifiés

ID	Risque	Impact potentiel	Probabilité	Mitigation
R-01	Faux positif répétitif	Désactivation par l'utilisateur	Moyen	Temporisation, seuil ajustable
R-02	Faux négatif critique	Observateur non détecté	Moyen	Communication des limites
R-03	Caméra indisponible	Système inutilisable	Faible	Transition Error, message clair
R-04	Modèle non téléchargé	Fallback Haar dégradé	Moyen	Téléchargement auto, fallback
R-05	Charge CPU élevée	Ralentissement du poste	Faible	Scan 500 ms, 640×480
R-06	Faux positif antivirus	.exe bloqué	Élevé	Signature de code (hors MVP)
R-07	Usurpation par photo	Tiers non détecté comme intrus	Faible	Pas de liveness dans le MVP
R-08	PIN non sécurisé	Code PIN lisible en JSON	Moyen	Hachage SHA-256 recommandé
R-09	Mauvaise interprétation	Fausse confiance absolue	Moyen	Communication des limites

Chapitre 16

Plan d'amélioration

16.1 Court terme (MVP+)

Hachage du code PIN (SHA-256 + sel). Tests unitaires des modules core et vision. Ajout d'un indicateur visuel de confiance de détection. Mode de diagnostic affichant les landmarks en surimpression. Raccourcis clavier globaux. Icône dans le system tray avec menu contextuel.

16.2 Moyen terme (v3.0)

Intégration de liveness detection (anti-spoofing). Chiffrement AES-256 des fichiers de profil. Mode multi-écrans. Notice d'information RGPD intégrée. Analyse d'impact (AIPD). Tests automatisés (pytest, Playwright). Signature de code Windows (certificat EV) et macOS (notarisation). Internationalisation anglais/français/allemand.

16.3 Long terme (v4.0+)

Eye-tracking de précision via modèle CNN dédié. Mode entreprise avec politique de groupe (GPO). Tableaux de bord d'administration IT. Certification ISO 27001. Conformité WCAG 2.1 niveau AA. Mise à jour automatique avec vérification de signature. SDK pour intégration dans des applications tierces.

Chapitre 17

Stratégie de validation

TABLE 17.1 – Plan de tests

ID	Scénario	Conditions	Résultat attendu	Type
TV-01	Visage unique	1 utilisateur face caméra	Détecté, score = 0, état ACTIVE	Nominal
TV-02	Second visage	1 tiers s'approche	Score augmente, protection déclenchée	Nominal
TV-03	Fausse alerte	Utilisateur rejette	Score remis à 0, état ACTIVE	Nominal
TV-04	Partage 60 s	Utilisateur accorde partage	Protection levée 60 s, réactivation	Nominal
TV-05	Absence	Aucun visage 15+ scans	Overlay cadenas/PIN	Nominal
TV-06	PIN correct	Bon code saisi	Protection levée	Nominal
TV-07	PIN incorrect	Mauvais code	Message erreur, écran verrouillé	Dégradation
TV-08	Caméra déconnectée	Webcam retirée	Transition ERROR	Dégradation
TV-09	Éclairage faible	Pièce sombre	Détection dégradée	Dégradation
TV-10	MediaPipe absent	Pas de modèle, pas d'internet	Fallback Haar, fonctionnalité réduite	Dégradation

Chapitre 18

Conclusion

ScreenGuard2 constitue un MVP fonctionnel et crédible d'application de protection visuelle contextuelle de l'écran. Il démontre la faisabilité technique d'un pipeline complet — de la capture webcam à la protection visuelle — entièrement local, sans dépendance cloud, avec une latence compatible avec un usage interactif et une interface utilisateur de qualité premium.

Le projet atteint ses objectifs de démonstration sur les plans suivants : détection faciale multi-visages via MediaPipe Face Mesh, estimation heuristique du regard via les landmarks de l'iris, identification de l'utilisateur principal par encodage facial calibré, scoring de suspicion progressif avec temporisation anti-faux positifs, protection visuelle différentielle (observateur vs absence), partage temporaire avec réactivation automatique, verrouillage par code PIN, et interface JARVIS HUD avec bridge bidirectionnel Python–JavaScript.

Positionnement du projet

Il est essentiel de positionner ce projet pour ce qu'il est réellement : un **prototype avancé de niveau MVP**, et non un produit industrialisé. L'estimation du regard est heuristique. La reconnaissance faciale n'est pas une biométrie certifiée. Le code PIN est stocké en clair. L'exécutable n'est pas signé. Aucun test automatisé n'est présent. La conformité RGPD est intentionnelle mais non formellement validée.

Dans ces conditions, ScreenGuard2 constitue une **base technique solide** pour un développement produit ultérieur, sous réserve d'une industrialisation comprenant : tests automatisés, chiffrement des données sensibles, signature de code, analyse d'impact RGPD, tests utilisateurs, et certification de sécurité.

Le chemin du MVP au produit est clairement identifié et documenté dans ce dossier.